1. Introduction

- **What is RAG**: Retrieval-Augmented Generation (RAG) is an architectural pattern that optimizes the output of a Large Language Model (LLM) by referencing a specific, authoritative knowledge base outside of its initial training data.
- **Why it is Needed**: Standard LLMs often suffer from "hallucinations" or outdated information. RAG ensures the model remains grounded in verified company documents, providing factually accurate and up-to-date responses.
- **Use Case Overview**: This system is designed as a **Customer Support Bot**. It automates responses to common queries using a product PDF while escalating complex or "out-of-scope" questions to human agents to maintain service quality.

---

2. System Architecture Explanation

- **Detailed Explanation of HLD**: The architecture is a modular pipeline consisting of an **Ingestion Layer** (PDF processing), a **Storage Layer** (ChromaDB), and an **Orchestration Layer** (LangGraph).
- **Component Interactions**:
    1. The **Ingestion Module** populates the **Vector Store**.
    2. The **FastAPI Interface** receives user queries and triggers the **Graph Workflow**.
    3. The **Graph** coordinates between the **Retriever** and the **LLM** to finalize the state.

---

3. Design Decisions

- **Chunk Size Choice**: A chunk size of **500 characters** with a **50-character overlap** was chosen. This ensures that text segments are small enough for precise retrieval but large enough to maintain semantic meaning.
- **Embedding Strategy**: We utilized the all-MiniLM-L6-v2 model. This strategy balances high-dimensional semantic accuracy with low-latency local execution.
- **Retrieval Approach**: A **top-K (K=3)** similarity search was implemented. This provides sufficient context for the LLM without exceeding token limits or introducing irrelevant noise.
- **Prompt Design Logic**: The prompt uses **Strict Contextual Grounding**. It instructs the LLM to reply with a specific keyword ("ESCALATE") if the answer is not found in the provided context, preventing hallucinations.

---

4. Workflow Explanation

- **LangGraph Usage**: LangGraph is used to define the system as a **state machine**. This allows for non-linear logic, such as looping back for human input or conditional exit paths.
- **Node Responsibilities**:
  - **Retrieve Node**: Queries the vector database and populates the context.
  - **Processing Node**: Analyzes context and generates the initial response.
  - **Human Node**: Pauses execution to accept manual agent input.

- **State Transitions**: Data flows via a **State Object** (AgentState) containing the query, context, and response. Transitions are managed by a central router that evaluates the category flag.

---

5. Conditional Logic

- **Intent Detection**: The LLM evaluates if the user's query matches the information available in the retrieved chunks.
- **Routing Decisions**:
  - **Support Path**: If context is sufficient, the system moves to END.
  - **Escalation Path**: If context is missing or the query is too complex, the system routes to the Human Node.

---

6. HITL Implementation

- **Role of Human Intervention**: Human-in-the-Loop (HITL) acts as the final authority. It ensures that "Unique" or "Critical" queries receive 100% accurate human-verified answers.
- **Benefits and Limitations**:
  - **Benefits**: Eliminates incorrect AI answers and builds user trust.
  - **Limitations**: Increases response time for escalated queries as it depends on human availability.

---

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed**: Increasing the number of retrieved chunks (K) improves accuracy but increases LLM processing time and cost.
- **Chunk Size vs. Context Quality**: Larger chunks provide more context but can dilute the specific answer, whereas smaller chunks are precise but may lack surrounding meaning.
- **Cost vs. Performance**: Using a high-performance cloud LLM (Groq/Llama 3) provides superior speed but requires external API calls, unlike a fully local offline model.

---

8. Testing Strategy

- **Testing Approach**: We utilized a **System Integration Test** using the FastAPI Swagger UI. We tested the "Happy Path" (queries in the PDF) and the "Edge Case Path" (queries requiring escalation).
- **Sample Queries**:
    - *In-Scope*: "What is the frontend architecture?"
    - *Out-of-Scope*: "How do I bake a chocolate cake?"

---

9. Future Enhancements

- **Multi-document Support**: Expanding the ingestion.py module to handle entire directories of PDFs.
- **Feedback Loop**: Allowing users to rate answers to fine-tune the retriever's performance over time.
- **Memory Integration**: Adding conversational memory so the bot remembers previous questions in a session.
- **Deployment**: Containerizing the application using **Docker** for deployment on AWS or Azure.